

Course Title	Craft Of Computing		
Programme Title	B.Sc. Information Sciences		
Mode	M1	Level	1
Course ID		Credits	5
Course Type	Core	Semester	3
Version	1.0	Academic Year	2026-27
Course Development Team	Anuradha, SriRam Last Edited: Jerome		

Description

Craft of Computing is a 5-credit course in the BSc Information Sciences Program. It is a preparatory course for the IS major and therefore it is structured to give a sense of the field at a bird's eye view level. Students see snippets of what they can expect as a part of the curriculum: some mathematics, samples of tools and technology, computer internals with a taste of its underlying scientific nature, We also present its historical and philosophical context. It is designed to be a breadth-first sweep over the discipline - a holographic view of some of the elements that constitute Information Sciences.

Rationale and Introduction

A key to effective computing is in establishing the right relation between human and machine so that usual chores that require labour are systematically automated. This process is more involved than meets the eye. It requires experience with designing small machines, so that innards of larger machines are understood with clarity. It requires a reflection on our own thinking processes and therefore problem-solving methods, so that we have a clarity of what needs to be automated and how to make the machine mimic our

methods. It requires a formal language for dialoguing with the machine and interpreting its results.

This course has been designed to touch upon all these aspects of human-machine interactions and give students a glimpse of the kind of width and depth that constitutes an information scientist – one who can convert life problems into technological solutions. However, given that beginners are often intimidated by technology, we weave the course through fun sessions with Arduino kits, teamwork, debates and presentations so that the psychological hurdles disappear into the light form of the course. We achieve this without compromising rigor or technical content, thus providing a panoramic experience ranging from computer internals to layers of software tools.

Intended Learning Objectives

- Design and construct simple hardware and software artifacts.
- Reflect on the nuances of a problem and design a solution for it.
- Classify Technical content, organize and explain it.

Prerequisites

None.

Syllabus and Readings

Craft of Computing is designed with 4 units: Unit 1 gives a taste of the world of machines; Unit 2 is a navigation through the Linux ecosystem as a preparation for future courses and Unit 3 is a gentle, very brief exposure to programming with *python*. Unit 4 is about the what and how of data. These units also provide the context for dovetailing historical and philosophical foundations of the discipline.

	Unit name	Weeks
Unit 1	Crafting Little Machines	3.5
Unit 2	The Operating System as an Ecosystem	4
Unit 3	A Peek at Programming	4
Unit 4	Introduction to Data-ing	2.5

Unit 1. Crafting Little machines (3 weeks)

This unit (3 weeks) is a hands-on exploration of the question: What is a machine?

What are its building blocks? How does electricity become electronics and from there digital?

Towards this, we show the assembly of preexisting Arduino demo projects, just joining up the layout, seeing what all goes into it, including the software that drives it. There is almost no or very little coding, just rewiring and making things work. They discover voltage, current relations, need for capacitors/resistors along the current path etc and the basics of the makeup of analog circuitry. We extend this to the context of a computer where again peripherals and analog sources of electricity are joined together to make a digital foundation for “logical” use. We then describe how pieces of “logic” join up into a high-level view of a computer’s internal organization.

Topics:

- Basic Electricity and Electronics (week 1)
- Arduino Playground, Tinker cad (week 2)
- Small Projects with Arduino kits (week 2, 3)
- Connecting Peripherals, Assembling a computer (week 3)
- Basic block organization of a computer(week 4)

Readings:

- Electricity Basics:
<https://web.pdx.edu/~rueterj/courses/ESM342-solar/WK3-GE-MC1-ElectricityBasics.pdf>
- Arduino Resource: <https://www.youtube.com/watch?v=TYMreY1NL78>
- Building a computer: <https://edex.adobe.com/teaching-resources/pc-assembly-and-disassembly>

General Readings:

- "Make Electronics: Learning by Discovery" by Charles Platts.
- "Getting Started with Arduino" by Massimo Banzi and Michael ShilohA .
- "Build Your Own PC" by Morris Rosenthal

Unit 2. The Operating System as an Ecosystem (4 weeks)

The operating system that as a layer over raw machines facilitates all the above data-ing and scripting learnt in the previous unit is the first takeaway from this Unit. A lot of exercises in class are towards “talking” to it and affecting changes in the way it organises and persists data.

Students are also prepared to navigate and structure their files and resources and script elementary “shortcuts” for repetitive work. *Topics*

- What is an OS? How is it structured? (Week 4)
- Files Directories: concept and pragmatics. (Week 5)
- Bash commands. (Week 5,6)
- Using an editor. (Week 6, 7)
- Basic shell configuration. (Week 7,8)
- Basic Administration, access rights, managing users. (Week 8)

Readings

- <https://github.com/trinib/Linux-Bash-Commands>
- <https://www.geeksforgeeks.org/introduction-to-linux-operating-system/>

- Learning the bash Shell, 3rd Edition, by [Cameron Newham](#) (chapters 1-10)

General Readings

- Learning Modern Linux, by [Michael Hausenblas](#)
- Linux in a Nutshell, 6th Edition, by [Ellen Siever](#), [Stephen Figgins](#), [Robert Love](#), [Arnold Robbins](#)
- Operating System Concepts, by Abraham Silberschatz, Greg Gagne, and Peter Baer Galvin

Unit 3: A Peek at Programming (4 weeks)

As an introduction to programming with emphasis on its 2-timing-ness [planning and execution], we extend isolated shell commands to be joined together into little tool-scripts. From here, we step into a programming environment with Python, discussing movements from the representations on a machine to abstractions in a programming/scripting environment. Programming is presented as constructive mathematics and shown to be the same as “instructing” a mathematical machine which in turn talks to a “real” machine. We hint at questions of good and bad ways, faster and slower ways of doing this.

Topics

- Bash programming to understand the place of tools (Week 9)
- Number Systems, Logic Gates (Week 9,10)
- Introduction to Python (Week 10 - 12)
 - Basic Computation: with List, Tuples
 - Branching, Iteration
 - Abstraction, Functions

Readings

- <https://www.freecodecamp.org/news/shell-scripting-crash-course-how-to-write-bash-scripts-in-linux/>
- “Structured Computer Organization” by Andrew Tanenbaum (chapter 1, sections of chapter 2)
- Python Tutorial : <https://docs.python.org/3/library/index.html>
- Book: <https://diveintopython3.net/>
- Python Tutor: <https://pythontutor.com/>

General Readings

- [A complete course on Linux Bash Shell Scripting with examples](#), by Imran Afzal
- <https://greenteapress.com/thinkpython2/thinkpython2.pdf>

Readings

- **The History of Computing: A Very Short introduction**, by Doron Swade
- <https://www.toppr.com/guides/computer-aptitude-and-knowledge/basics-of-computers/history-of-computers/>
- <https://www.iwm.org.uk/history/how-alan-turing-cracked-the-enigma-code>

Unit 4. Introduction to Data-ing (2.5 weeks)

Excel Sheets are useful tools to tabulate, calculate and track simple things like expenses, marks etc. Technically speaking, they provide a functional language, extending calculators to larger scales of data. In this Unit, we begin by examining a small slice of the Data world, with Excel sheets and graduate to writing macros to do the jobs we first did manually. We then gently slide into a demo of the world of Programming with Python Tables doing the same job. Not much programming is learnt here but the sense of programming as a 2-time-activity -- planning and execution – emerges from this.

Topics

- Using Excel sheets (Week 12)
- Using special excel functions (Week 13)
- Writing little macros (Week 13, 14)
- A sense of AI powered data-ing(Week 14)

Readings

- Teach Yourself VISUALLY Excel 2019" by Paul McFedries (Chapters 1-10)
- <https://www.excel-easy.com/vba/create-a-macro.html>

General Readings

- The Excel 2016 Bible, by [Jordan Goldmeier](#)

Week-wise Schedule

Unit Name	Week	Class Plan
Unit 1	1, 2,3,4	● Basic Electricity and Electronics ● Arduino Playground, Tinker Cad Simulations ● Small Projects with Arduino kits ● Connecting Peripherals, Assembling a computer , Block organization of a computer
Unit 2	5,6	● What is an OS? How is it structured? ● Files, Directories ● Bash commands
	6, 7	● ● Using an editor, Basic shell scripting - Basic Administration, access rights, managing users
	8	● Basic shell configuration. (Week 7,8) ● Basic Administration, access rights, managing users.

Unit 3	9,10	• Bash programming to understand the place of tools • Number Systems, Logic Gates
	11,12	• Introduction to Python (Week 10 - 12) ◦ Basic Computation: with List, Tuples ◦ Branching, Iteration ◦ Abstraction, Functions
Unit 4	13	• Using excel sheets • Using special excel functions
	14	• Excel macros • AI powered data-ing
	15,16	• Assessment Week: Project Presentations

Pedagogy

This preparatory course is taken by beginners who need to be motivated to get beyond their usual use of computers towards becoming IS majors. Towards that, we need to provide motivation and a sense of the field together.

We achieve this by the following Pedagogical approach:

- A hands-on course - both with hardware and software.
- Form and Content both deliver content. Conceptual ideas are dovetailed with lab work so there is a sense of it being one and the same activity.
- Problem based learning: We choose unconventional problems that are fun for young people so that technology, science and even philosophy can be sneaked in as it were in the middle of an exploration.
- Team-based learning: Students work in groups to make small Arduino projects or exploratory projects that will use the core conceptual ideas and also use software tools to depict this.

Assessment

The assessment methodology for the course is summarized in the table below.

Assessment Type	Number of assessments in the semester	Contribution to overall score
Quizzes (Best 4 out of 5)	5	40%
Take-home individual assignments	2	10%
Group Project	2	40%
Class participation	1	10%

- **Quizzes:** There will be totally 5 in-class quizzes.
Quiz 1: Will cover materials covered from basic electricity to basic Arduino.
Quiz 2: Will cover basics of VI.
Quiz 3: Will cover Linux commands.
Quiz 4: Will cover basics of number systems and logic gates.
Quiz 5: Will cover Python programming.
- **Assignments/Problem Sheets:** There will be either two take home assignments/problem sheets each contributing 5% towards the final grade
- **Group Projects**
There are two compulsory projects that students undertake to do in groups. The first project will be based on Arduino hardware (submission before mid-semester break) and one exploratory project on Excel and Linux (submission toward final assessment week)
- **Class Participation**
Active learning is an essential part of this course. Therefore, class participation is an important assessment component. (**A** = Attendance percentage, **B** = Quality of answers and contributions, **C** = Responsiveness, preparation, and aptitude for learning, **D** = Attitude, class conduct, and behaviour)
Score = $(0.2A + 0.4B + 0.3C + 0.1D)$

General Reading

- *The New Turing Omnibus: Sixty-Six Excursions in Computer Science*, A. K. Dewdney
Publisher: Henry Holt and Company, 2001, ISBN:0805071660, 9780805071665

- **Invitation to Computer Science, Paperback – Student Edition, 1 January 2018**
by Judith Gersting and G.Michael Schneider.